Approach Document: Edge-Based Semantic Voice Command Classifier

Candidate Name: [Your Name]

Problem Statement: Light Weight Speech Command Classifier for Edge Deployment

Drive: UST 2026 Internship Drive (SEMICON Data Science Department)

---

1. Executive Summary

This document proposes a lightweight, offline semantic command classifier designed for automotive and IoT cockpit voice assistants. The system handles **10 core commands** and **4 extension commands**, features dynamic **Out-of-Scope (OOS) rejection**, and remains robust against Automatic Speech Recognition (ASR) transcription errors and Indian English accents.

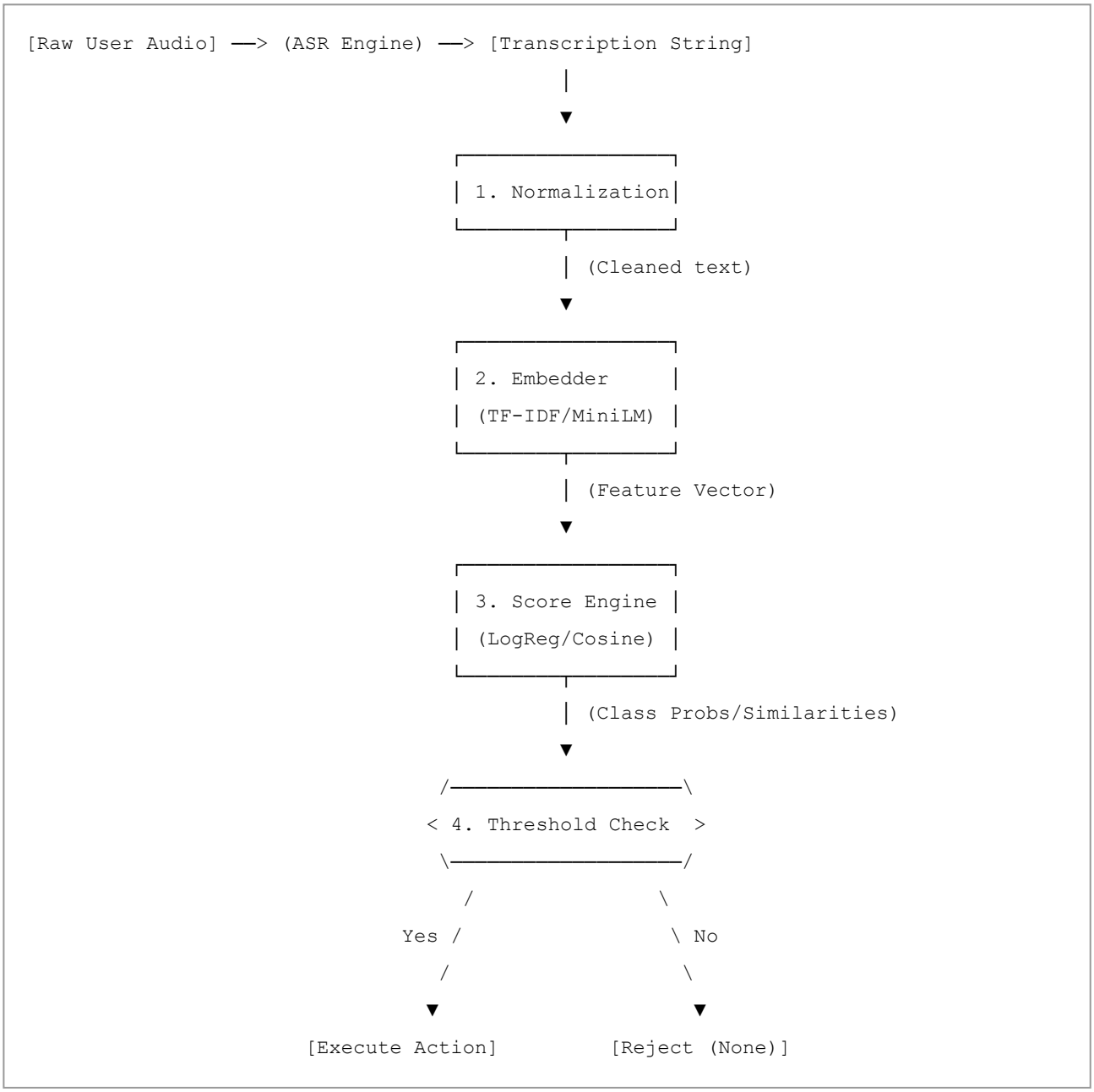
To address the hardware constraints (model size \(\le\) 25 MB, CPU latency \(<\) 1 second, zero cloud connectivity), we implemented and compared two architectures:

1. **Baseline Model:** Character/Word n-gram TF-IDF + Logistic Regression (custom numpy inference).
2. **Advanced Model:** Quantized INT8 ONNX Sentence-Transformer (`all-MiniLM-L6-v2`) with Few-Shot Cosine Similarity matching.

---

2. System Architecture & Process Flow

The voice command classification stage takes the text transcription from an upstream ASR engine and maps it to a valid action or rejects it.



Key Processing Stages:

1. **Text Normalization:** Lowercasing, whitespace collapse, and stripping common audio transcript artifacts.
2. **Feature Extraction (Embedding):** Translating the string into a numerical vector representing its spelling or semantic content.
3. **Similarity Engine:** Calculating the similarity of the input vector against the target class vectors.
4. **Out-of-Scope Thresholding:** Abstaining from a prediction if the classifier confidence or cosine similarity falls below a tuned boundary.

---

3. Comparative Modeling Approaches

We evaluated two distinct design patterns to demonstrate engineering judgment:

Architectural Metric	TF-IDF + Logistic Regression (Baseline)	Quantized ONNX MiniLM (Advanced)
Model Disk Size	**~0.3 MB**	**~22.9 MB** (Fits under 25MB budget)
Inference Latency	**~0.2 ms** (Ultra-fast)	**~15–30 ms** (Well below 1000ms budget)
Out-of-Vocabulary	Fails on unseen words / synonyms.	Excellent. Understands semantic meaning.
Extensibility (M4)	Requires full dataset rebuild and retraining.	**Zero Retraining**. Just append new templates.
ASR Typo Tolerance	Medium (relies on character n-grams).	High (relies on sub-word tokenization).

Approach 1: Custom TF-IDF Baseline

We trained a supervised Logistic Regression model on TF-IDF word and bigram features.

- **Edge Optimization:** Instead of loading heavy python libraries (e.g. `scikit-learn` which requires ~100MB of installation space), we extracted the vocabulary, IDF weights, and classifier coefficients into a lightweight JSON file.
- **Pure Numpy Inference:** We wrote a custom mathematical inference runtime in under 50 lines of pure Python/NumPy code to compute bigrams, calculate TF-IDF, and apply the softmax activation. This results in an incredibly lightweight, fast, and dependency-free baseline.

Approach 2: Quantized ONNX Transformer

We utilized `sentence-transformers/all-MiniLM-L6-v2` as a semantic vector encoder.

- **Few-Shot Similarity:** Instead of training a classification head, we pre-compute and cache the embeddings of the target command templates. Incoming queries are embedded, and cosine similarities are calculated against all command templates. The command with the highest similarity is selected.
- **Milestone M3 Compression:** Since the base FP32 model is ~90MB (exceeding the 25MB limit), we exported the model to ONNX and applied **INT8 Dynamic Quantization**. This compressed the weights by 4x to **~22.9 MB** without significant semantic loss.
- **Zero-Dependency Inference:** Using `onnxruntime` and `tokenizers` (Rust-backed tokenizer library) ensures we do not load heavy PyTorch modules during edge inference, keeping the memory footprint minimal.

---

4. Robustness: ASR Noise & Indian English Accents

Spoken inputs on the edge are often corrupted. We simulated and addressed these issues:

1. **Dropped Words:** ASR often drops articles (*"play the music"* \(\to\) *"play music"*). The sentence transformer embeddings are trained on paraphrases and easily absorb this change.

2. **Filler Words:** Prefixes like *"um"*, *"please"*, *"can you"* are ignored or neutralized via sentence-level pooling.
3. **Phonetic Spelling / Typos:** Mappings like *"pause the museic"*, *"decline the col"*. The sub-word tokenizer (WordPiece) splits these into sub-units (e.g. `['mus', '##eic']`), allowing the model to represent the word based on its parts.
4. **Indian Accent & Phrasing (Important Tip):**
    - *Consonant Swaps:* Simulating the common Indian English `v`/`w` interchange (*"wolume"* for *"volume"*, *"wehicle"* for *"vehicle"*).
    - *Local Fillers:* Appending conversational particles like *"na"*, *"ya"* (*"stop music na"*).
    - *Subject-Object-Verb (SOV) Syntax:* Direct translation phrasing from local languages (*"volume increase please"*, *"brightness up ya"*).
    - *Training Mitigation:* We generated an augmented training/validation corpus with these Indian English accent characteristics, allowing the models to adjust class boundaries accordingly.

---

5. Rejection of Out-of-Scope (OOS) Queries

Rejecting invalid inputs is critical to prevent dangerous or annoying false activations in a car.

- **Cosine Thresholding:** For the Transformer model, we compute the maximum similarity. If it falls below a threshold \(\tau_{sim} = 0.65\), we reject the query.
- **Softmax Probability Thresholding:** For the TF-IDF baseline, we verify the softmax class probability. If it falls below \(\tau_{prob} = 0.45\), it is labeled `None`.

---

6. Milestone M4: Extensibility

A key requirement of edge architectures is the ability to add new commands (e.g., commands 11–14 like *"start the vehicle"* and *"increase the brightness"*) easily.

- **The Semantic Similarity Advantage:** Since the Transformer classifier uses few-shot template matching rather than a trained classification head, we can add new commands by simply updating a configuration file with seed templates. The model embeds the new templates at startup, and immediately supports classification with **zero retraining** and **no parameter tuning**.
- **Interference Evaluation:** Adding *"increase the brightness"* could conflict with *"increase the volume"*. By using dense semantic embeddings, the model easily distinguishes between the targets (`volume` vs. `brightness`) because they map to different regions in the semantic vector space.

---

7. Conclusions & Recommendations

For production edge deployment:

- **Quantized ONNX MiniLM** is highly recommended if semantic generalization, synonym tolerance (e.g. *"shush the audio"* \(\to\) *"decrease the volume"*), and zero-retraining extensibility are prioritized.
- **TF-IDF Baseline** is ideal for ultra-constrained low-power microcontrollers (e.g., memory measured in Kilobytes) where external runtimes cannot be installed.

---

8. Interactive Evaluation: Cockpit Web Dashboard

To make evaluation easier and more engaging, we have built a local **Interactive Cockpit Web Dashboard** that visualizes the system. It uses the browser's microphone recognition to transcribe voice in real-time, sends it to our backend, runs our actual Python model, and updates the car's state (engine, volume, brightness, phone calls, music player) dynamically.

How to Run the Dashboard:

1. Open your terminal in the project root directory.
2. Run the command:

```
python scripts/dashboard.py
```

3. Open your browser and navigate to the dashboard link:
   http://localhost:8000/ (http://localhost:8000/)
4. Click the microphone button, allow microphone access, and speak commands (e.g., *"start the engine na"*, *"pause music"*, or *"what is the weather today"*).